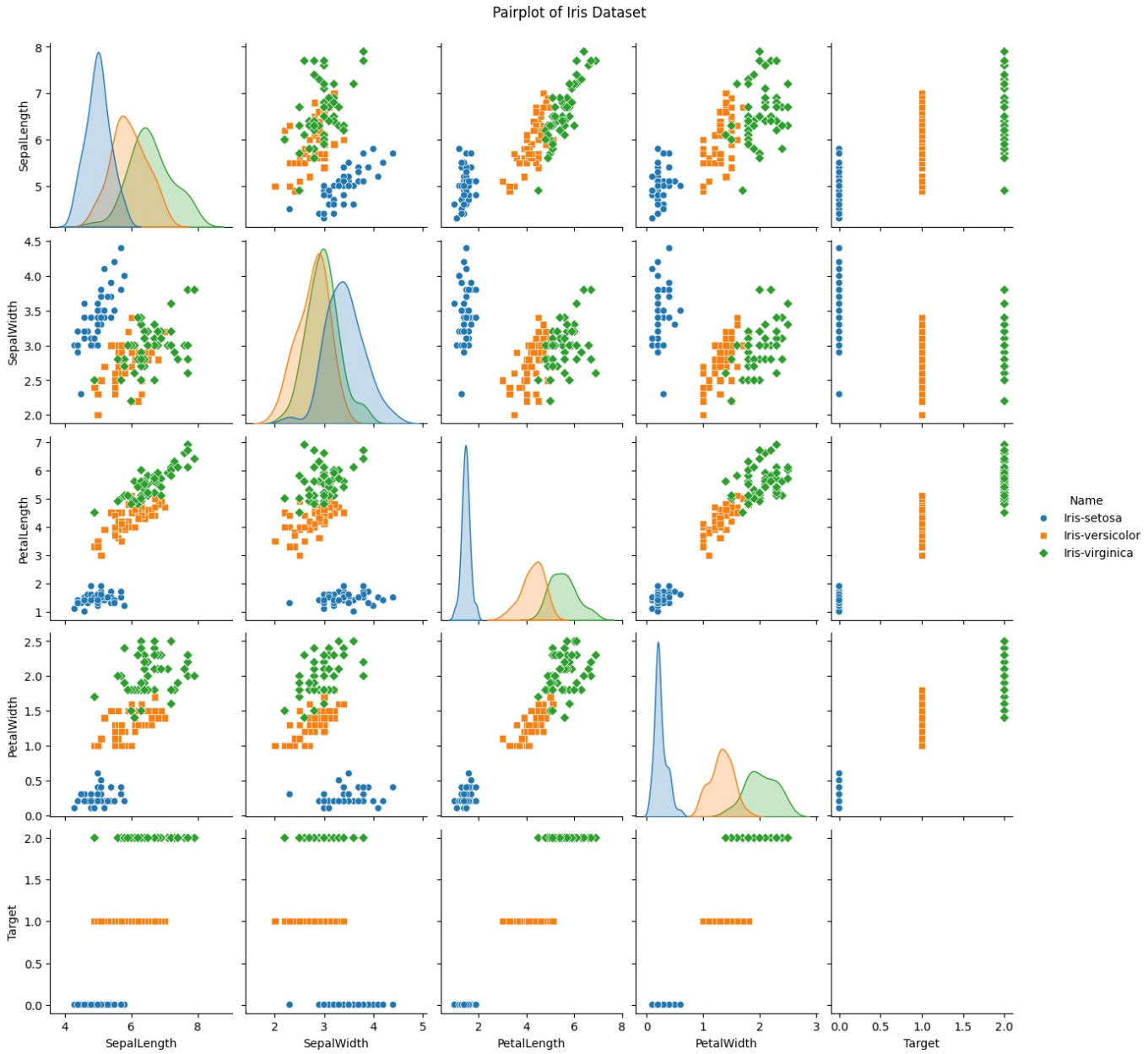
In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.neighbors import KNeighborsClassifier
from collections import Counter
```

In [2]:
```
# Load the dataset
df = pd.read_csv('iris.csv')
df.head()

# Dataset Information
print("\nDataset Info:")
df.info()

# Encode Target Labels
class_mapping = {label: idx for idx, label in enumerate(df['Name'].unique())}
df['Target'] = df['Name'].map(class_mapping)

print("\nClass Mapping:", class_mapping)
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   SepalLength  150 non-null    float64
 1   SepalWidth   150 non-null    float64
 2   PetalLength  150 non-null    float64
 3   PetalWidth   150 non-null    float64
 4   Name         150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

Class Mapping: {'Iris-setosa': 0, 'Iris-versicolor': 1, 'Iris-virginica': 2}
```

In [3]:
```
# Pairplot to visualize the data distribution
sns.pairplot(df, hue='Name', markers=["o", "s", "D"])
plt.suptitle("Pairplot of Iris Dataset", y=1.02)
plt.show()
```



In [4]:
```
# Features and Target
X = df.drop(['Name', 'Target'], axis=1).values
y = df['Target'].values

# Split data: 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(f"Training set size: {X_train.shape[0]}")
print(f"Testing set size: {X_test.shape[0]}")
```

```
Training set size: 120
Testing set size: 30
```

KNN Implementation (Euclidean Distance)

In [5]:
```
def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))

class CustomKNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        predictions = [self._predict(x) for x in X]
        return np.array(predictions)

    def _predict(self, x):
        # Compute distances
        distances = [euclidean_distance(x, x_train) for x_train in self.X_train]
```

```
        # Get k nearest samples, labels
        k_indices = np.argsort(distances)[:self.k]
        k_nearest_labels = [self.y_train[i] for i in k_indices]

        # Majority vote
        most_common = Counter(k_nearest_labels).most_common(1)
        return most_common[0][0]

# Train Custom KNN
k_value = 5
custom_knn = CustomKNN(k=k_value)
custom_knn.fit(X_train, y_train)

# Predict and Evaluate
custom_preds = custom_knn.predict(X_test)
custom_accuracy = accuracy_score(y_test, custom_preds)
custom_cm = confusion_matrix(y_test, custom_preds)

print(f"Custom KNN Accuracy: {custom_accuracy * 100:.2f}%")
print("\nClassification Report:\n", classification_report(y_test, custom_preds, target_names=class_mapping.keys
```

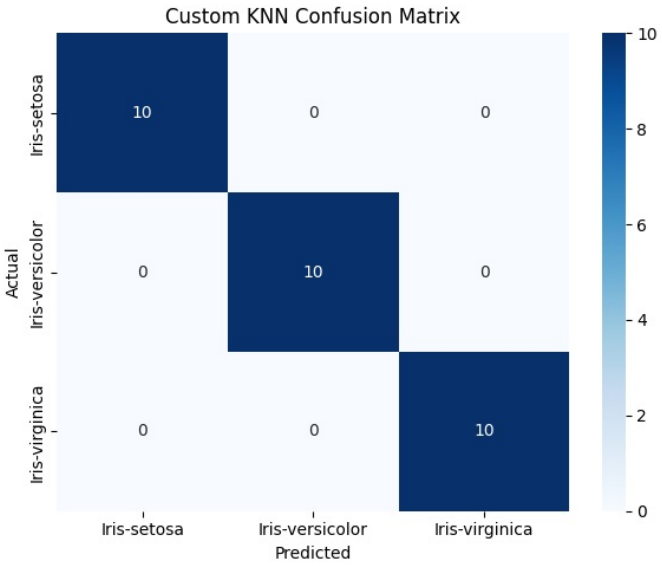
```
Custom KNN Accuracy: 100.00%

Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00        10
 Iris-virginica       1.00      1.00      1.00        10

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

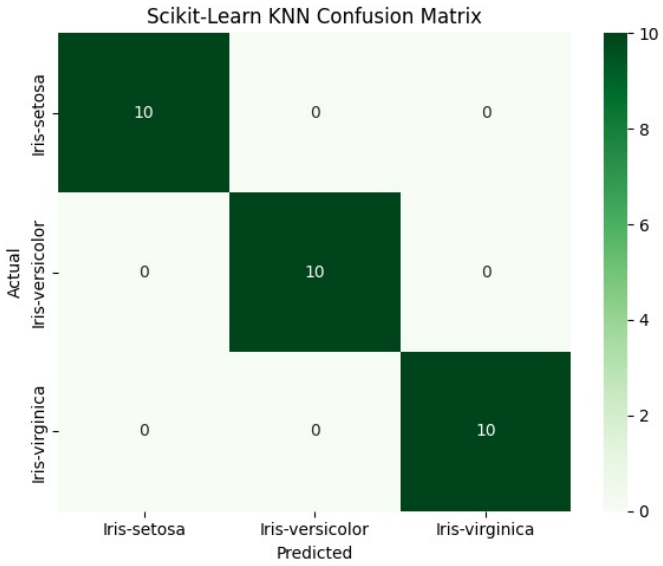
. Scikit-learn KNN Implementation

In [6]:
```
# Initialize and train Scikit-Learn KNN
sklearn_knn = KNeighborsClassifier(n_neighbors=k_value, metric='euclidean')
sklearn_knn.fit(X_train, y_train)

# Predict and Evaluate
sklearn_preds = sklearn_knn.predict(X_test)
sklearn_accuracy = accuracy_score(y_test, sklearn_preds)
sklearn_cm = confusion_matrix(y_test, sklearn_preds)

print(f"Scikit-learn KNN Accuracy: {sklearn_accuracy * 100:.2f}%")
print("\nClassification Report:\n", classification_report(y_test, sklearn_preds, target_names=class_mapping.keys
```

```
Scikit-learn KNN Accuracy: 100.00%

Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00        10
 Iris-virginica       1.00      1.00      1.00        10

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

In [7]:
```
# 1. Confusion Matrix Comparison
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.heatmap(custom_cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=class_mapping.keys(), yticklabels=class_mapping.keys())
axes[0].set_title('Custom KNN Confusion Matrix')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

sns.heatmap(sklearn_cm, annot=True, fmt='d', cmap='Greens', ax=axes[1],
            xticklabels=class_mapping.keys(), yticklabels=class_mapping.keys())
axes[1].set_title('Scikit-Learn KNN Confusion Matrix')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

plt.tight_layout()
plt.show()
```





In [8]:

```
# 2. Accuracy Comparison Bar Chart
models = ['Custom KNN', 'Scikit-Learn KNN']
accuracies = [custom_accuracy, sklearn_accuracy]

plt.figure(figsize=(8, 5))
bars = plt.bar(models, accuracies, color=['#3498db', '#2ecc71'], width=0.5)
plt.ylim(0, 1.1)
plt.title('Accuracy Comparison')
plt.ylabel('Accuracy Score')

# Add values on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.02, f'{yval*100:.2f}%', ha='center', va='bottom', fontsi

plt.show()
```